

dr. Matej Šprogar

Obravnavava napak

Kaj narediti z napakami v času izvajanja programa?

Program mora delovati "pravilno" tudi v primeru napak. Učinek napake je lahko ali vpadljiv (npr. očitno napačen rezultat, sesutje programa...) ali nevpadljiv (npr. napaka zaokroževanja, neizvršitev nekih zalednih operacij...).

Kode nikoli ne moremo napisati tako, da bi VSE napake lahko zaznal že prevajalnik, saj mora program preživeti tudi napake, ki se bodo pojavile v času izvajanja. Te nastanejo ali zaradi napak v algoritmu ali v podatkih (slabi ali napačni podatki, napačna uporaba programa (kdo je "kriv" v tem primeru, uporabnik ali programer?)).

Napake so torej nepredvideni "dogodki" med izvajanjem programa.

Predpogoj != detekcija napake...

Predpogoj prepreči napako v **zasnovi** programa - predstavlja splošno trditev, ki je VEDNO resnična. Z njim programer ubesedi svoja pričakovanja, ki **morajo** biti izpolnjena *pred* nadaljevanjem izvajanja.

```
int povprecje(SeznamStevil list) {  
    assert list != null; // Java  
    ...  
}
```

Če pa lahko višji nivo uporabi funkcijo "narobe", potem ne moremo uporabiti predpogoja, ampak detekcijo (in signalizacijo) napak.

Možne napake zaznavamo z navadnim pogojem.

```
int povprecje(SeznamStevil list) {  
    assert list != null;  
    if (list.length == 0)  
        ?hm_kaj_pa_zdaj?  
    ...  
}
```

Uporaba predpogojev

Predpogoj med izvajanjem programa išče **vsebinske** napake, ki jih prevajalnik ne more najti.

Z njim testiramo tim. *invariante* - to so vedno resnične lastnosti, kar pomeni, da s predpogoji NE SMEMO preverjati vsebine uporabniških podatkov.

V ta namen Java/C#/C++ ponujajo ukaz/funkcijo **assert**.

Asserti se zaradi vpliva na hitrost izvajajo zgolj med testiranjem!

Če bi vse funkcije *vedno* preverjale vse, bi program pogosto preverjal isto stvar večkrat, kar bi ga občutno upočasnilo. Zato izvedemo preverjanje le v eni točki programa, potem pa se vsa nadaljna koda zanaša, da ni težav... Če temu ni tako seveda pride do katastrofe, zato pa namesto (ponovnega) preverjanja raje zapišemo pričakovanja funkcije!
Na primer: *v tej točki izvajanja programa funkcija pričakuje, da podano polje obstaja in ni prazno.*

Napaka v predpogoju prekine izvajanje programa in v toliko jo zazna tudi površno testiranje. Posledično predpogoji niso potrebni v *release* kodi in zato ker prevajalnik *assert*-e izključi iz *release* verzije programa zaradi hitrejšega delovanja, v predpogoj NE sodijo delovni ukazi! Sicer se bo *release* koda obnašala drugače kot testna koda!

Predpogoj zgolj ubesedi naše pričakovanje, kaj bi naj veljalo na izvajalni poti programa. Če pričakovanje pade je to znak, da smo program zastavili narobe.

Asserti...

Assert se preverja zgolj med testiranjem (Debug) programa, ko omogoči iskanje napak v zasnovi programa - npr. uporaba funkcije v napačnem trenutku.

Predpogoj ubesedi pričakovanje, da je napačno uporabo preprečil že višji nivo; v toliko je ponovno preverjanje nepotrebno!

Ker se izvaja zgolj med testiranjem kode, predpogoj **NE SME VPLIVATI** na delovanje preostale kode!

Predpogoj izraža *teoretično* pričakovanje, da je že višji nivo poskrbel za določene stvari (npr. preveril, da polje obstaja; da ima vsaj en podatek...). Ampak vsako teoretično predpostavko moramo preveriti tudi v *praksi*.

Ker bi naj predpostavka veljala VEDNO, jo preverjamo zgolj v času (izčrpnega!) testiranja programa, ki preveri resničnost predpostavke v praktičnih testnih primerih.

```
for (int i=0; i<tab.Length; ++i)
    izpisi(tab, i);

void izpisi(int[] tab, int index) {
    assert tab!= null && index<tab.Length;
    ...
}
```

V Javi vklopimo assert z opcijo -ea, C# pa v Debug načinu.

Predpogoji, popogoji, invariante...

Popogoj je kriterij, ki mora veljati ob ZAKLJUČKU ali ukaza ali celotne funkcije. Z njim se programer prepriča, da koda res naredi to, kar bi naj naredila. Enako kot predpogoj tudi popogoj ne sme vplivati na izvajanje programa.

Uporabimo lahko enako sintakso kot za predpogoj, le da ga postavimo ZA blokom...

```
{...}  
popogoj(kriterij);
```

Invariantni pogoj ostane NESPREMENJEN* tudi po izvajanju bloka kode. Je varovalka, ki zagotavlja, da ne pride do nehotenih učinkov. Lahko ga vključimo kot komentar, najbolje pa v obliki pred/po-pogoja ali *konstantnih* vrednosti...

```
Contract.Invariant(dan >= 1 && dan <= 31);  
Debug.Assert(ura <= 23);  
assert velikost() < kapaciteta();
```

```
// C# readonly/const, Java final  
final Jezik jezik = pridobi_jezik();
```

Zaznavanje napak

Zaznavanje napak je mogoče s pogojnimi stavki za vsak kritični del kode. Če želimo preprečiti ***nastanek*** napake, moramo še *pred* izvedbo uporabiti pogoj za vsak morebiti nevaren ukaz. Če želimo preprečiti ***širjenje*** napake po programu, moramo preveriti tudi *pravilnost* vsakega rezultata.

Oboje žal pomeni obsežno programiranje, ki popolnoma pokvari berljivost osnovne delovne kode.

Funkcija, ki "ustvari" napako, ne ve, kdo jo je klical, zakaj ji je zastavil nemogočo nalogo in kaj sploh so pametni izhodi iz zagate.

Funkcija, ki doživi in/ali zazna napako, lahko:

1. **Prekine** delovanje programa (brutalno prisili programerja, da čim prej doda ustrezno obravnavo napak in prepreči ponovno prekinitev programa)
2. Vrne vrednost, ki **predstavlja** napako (ni vedno izvedljivo, predvsem pa zahteva izrecno in dosledno obravnavo napak v klicajoči kodi)
3. Vrne napačno(!) vrednost **in** postavi program v nelegalno stanje (tudi to zahteva izrecno in dosledno obravnavo napak v klicajoči kodi)
4. Kliče posebno **error** funkcijo (ampak kako pa naj error funkcija ve, kdo je povzročil napako in kako jo sanirati? In smo nazaj pri 1 | 2 | 3 ...)
5. Sproži tim. **izjemo*** in upa, da bo izjema nekje ulovljena in razrešena.

Dve osnovni težavi klasične obravnave napak

Recimo, da funkcija *kalk* izračuna *double* vrednost *string* izraza. Pričakujemo naslednje:

```
preveri(42.0 == kalk("6*7"));  
preveri(Double.IsInfinity(kalk("42/0")));           // C#  
preveri(kalk("0/0").isNaN());                       // Java
```

1. kaj naj funkcija *kalk* vrne v primeru napake v izrazu?

```
preveri(? == kalk("6#7"));                           // 6#7 ni NaN!
```

2. *klicajoča* koda mora VEDNO preveriti možnost napake!

```
if (veljavno(kalk("6*7"))) { ... }                   // komplicirano!
```

Prva težava: kaj vrniti?

Na eno vrednost omejen *return* ni idealen za vračanje napake: recimo `kalk("6#7")` ne more vrniti NaN, saj to pomeni nekaj drugega.

Razen če bi lahko vrnili dve vrednosti hkrati: rezultat in status napake!

To znamo na vsaj* tri načine:

- (A) z *globalnim* 🙄 statusom napake,
- (B) z objektom, ki hrani vrednost in še status vrednosti!
- (C) z *nullable* realnim številom (Java tip "Double", C# "double?").
- (D) ...

```
enum Status { OK, ...};
Status status;
double vrednost = kalk("6*7");           // (A)
if (status != Status.OK)
    obravnavaj_napako(status);
else
    normalno_naprej(vrednost);

class Odgovor {
    double vrednost;
    Status status;
};
Odgovor odgovor = kalk("6*7");           // (B)
if (odgovor.status != Status.OK)
    obravnavaj_napako(odgovor.status);
else
    normalno_naprej(odgovor.vrednost);

Double odgovor = kalk("6*7");             // (C)
if (odgovor == null)
    obravnavaj_napako(odgovor);
else
    normalno_naprej(odgovor);
```

Druga težava: preveč preverjanja!

Da bi **klicajoča** koda sploh zaznala napako, mora VEDNO preveriti status operacije. Če je prišlo do napake in če ve kako, lahko tudi ustrezno odreagira (recimo zahteva ponoven izračun, logira napako...), ali pa posreduje napako na še višji nivo.

Če klicatelj ne zna rešiti napake, jo **mora** posredovati na višji nivo!

Posledično je v klicajoči kodi veliko ali vsaj nekaj obravnave napak, kar občutno zmanjša berljivost kode in je hkrati morebiten vir novih napak!

```
string izraz = pridobi();
Double odgovor = kalk(izraz);

if (odgovor == null) {
    System.out.println("nekaj narobe?");
    if (posebni_pogoj(izraz))
        odgovor = 0;    // razrešimo
    else
        // ne znamo/smemo/moremo naprej
        System.exit(NEZNANO);
}

return normalno_naprej(odgovor);
```

Koda za obravnavo napak nesrečno "razdeli" *glavno* kodo in jo s tem naredi manj razumljivo...

Obravnava napak pokvari berljivost kode...

```
enum ErrorStatus {PREDOLGI_IZRAZ, MANJKA_OPERATOR, PRAZEN_IZRAZ, ...itd...};  
ErrorStatus error_status; // globalna spremenljivka
```

```
Integer isci_operator(String izraz) {  
    assert izraz != null;  
    if (izraz.length() > MAX_LENGTH) {  
        error_status = ErrorStatus.PREDOLGI_IZRAZ; // sporočilo za visji nivo  
        return null; // vrnitev na visji nivo  
    }
```

```
    final String veljavni_operatorji = "+-*/^";  
    final int pozicija_operatorja = najdi_prvega(izraz, veljavni_operatorji);  
    if (pozicija_operatorja == -1) {  
        error_status = ErrorStatus.MANJKA_OPERATOR;  
        return null;  
    }
```

```
    return pozicija_operatorja;
```

```
}
```

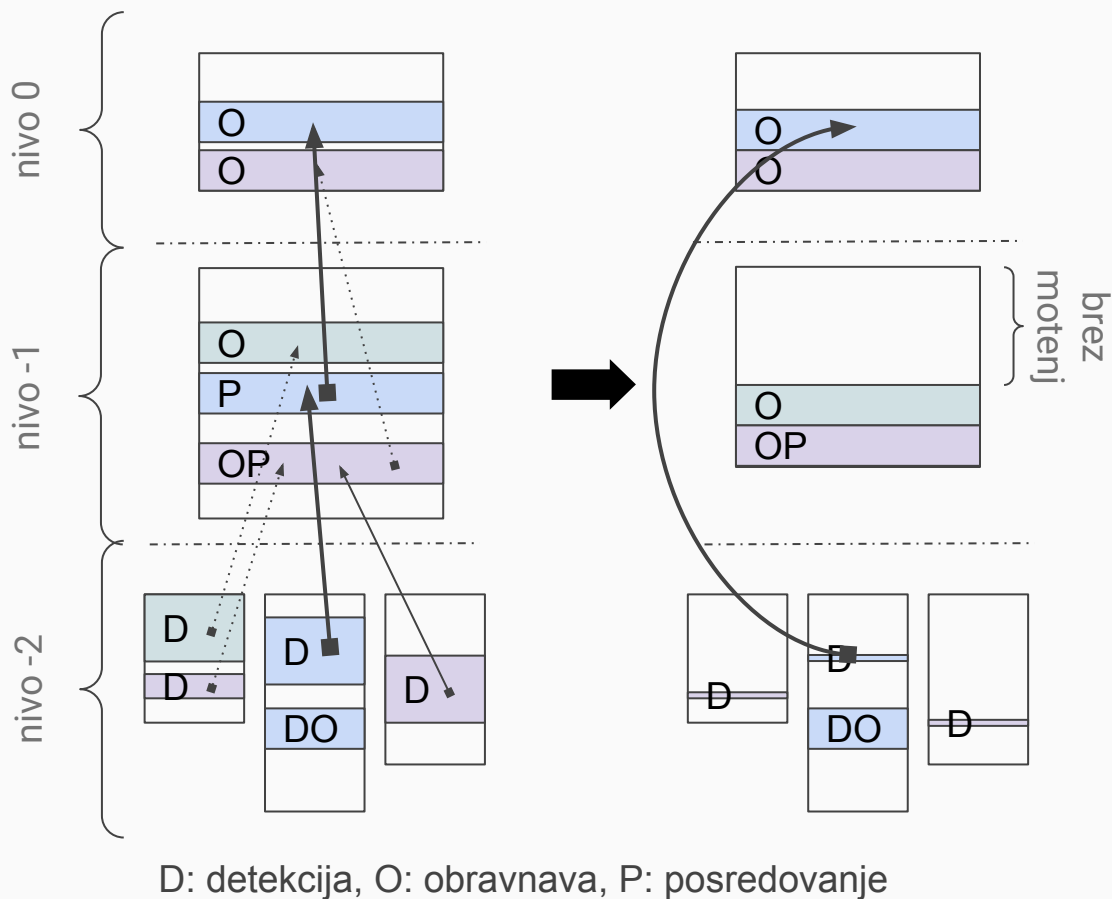
Kaj želimo?

Funkcija lahko zazna napako, vendar je ne zna vedno rešiti; obratno si klicatelj želi čim manj motenj v osnovni kodi.

Želimo torej:

1. enostaven mehanizem prenosa informacije o napaki iz nižjega na višji(e) nivo(je), in
2. razmejitev na kodo, ki opravlja osnovno funkcionalnost, in kodo, ki obravnava napake.

Oboje nam nudijo **izjeme*** (*exceptions*), ki pa imajo spet svoje probleme...



Pri združevanju kode
iz več virov **morajo**
vsi moduli uporabljati
ISTO strategijo
obravnavne napak